

The Impact of Parameters Optimization in Software Prediction Models

Asad Ali

University of Salerno

University of Salerno, Fisciano, Italy

Abasyn.asad@gmail.com

Carmine Gravino

University of Salerno, Fisciano, Italy

gravino@unisa.it

Abstract—Several studies have raised concerns about the performance of estimation techniques if employed with default parameters provided by specific development toolkits, e.g., Weka. In this paper, we evaluate the impact of parameter optimization with nine different estimation techniques in the Software Development Effort Estimation (SDEE) and Software Fault Prediction (SFP) domains to provide more generic findings of the impact of parameter optimization. To this aim, we employ three datasets from the domain of SDEE (China, Maxwell, Nasa) and three different regression-based datasets from the SFP domain (Ant, Xalan, Xerces). Regarding parameter optimization, we consider four optimization algorithms from different families: Grid Search and Random Search, Simulated Annealing, and Bayesian Optimization. The estimation techniques are: Support Vector Machine, Random Forest, Classification and Regression Tree, Neural Networks, Averaged Neural Networks, k-Nearest Neighbor, Partial Least Square, MultiLayer Perceptron, and Gradient Boosting Machine. Results reveal that, with both SDEE and SFP datasets, seven out of nine estimation techniques require optimization/configuration of at least one parameter. In majority of the cases, the parameters of the employed estimation techniques are sensitive to the optimization of specific types of data. Moreover, not all the parameters need to be optimized as some of them are not sensitive to optimization.

Index Terms—estimation techniques, parameters optimization, software effort estimation, software fault prediction

I. INTRODUCTION

To develop a high-quality software system (non-defective) and deliver it within scheduled budget and time, the Software Quality Assurance (SQA) team of an organization needs to focus on the Software Development Effort Estimation (SDEE) and Software Fault Prediction (SFP) based on the data of the previously completed project [1]. However, the amount of software project data is usually very huge and to analyze the data we need miners (e.g., machine learning-based estimation techniques) [2].

When using estimation techniques there are certain types of tunable parameters that depict the behavior and control the overall performance of the built prediction models. For instance, width of Gaussian kernels (sigma) is one of the SVM (Support Vector Machine) parameters which needs to be initialized to a particular value, and that value, more or less, characterizes the accuracy of SVM in a particular domain. Similar considerations also apply to other techniques such as Random Forest (e.g., number of decision trees) and Neural Networks (e.g., number of neurons in hidden layers).

To a large extent, researchers in Software Engineering believe that optimal and perfect settings/tuning of these parameters are not easy and hence they hesitate to tune them and prefer to keep going with the default settings provided by development toolkits, such as R, Weka, Python. For instance, according to [3], about 80% of the 50 most highly cited papers related to SFP prefer to use the default settings of various estimation techniques. Similarly, we performed a Systematic Literature Review (SLR) about SDEE using machine learning techniques [4], which considered the studies from 1991 to 2017 and, at the end, selected 75 final studies for the meta-analysis. Aiming to identify how many of them use optimization algorithms to configure the parameters of various estimation techniques, we have found that 88% of the 50 most citable studies preferred to use the default parameters settings.

One of the reasons the majority of the authors preferred to use default settings probably is that parameters optimization requires a substantial amount of time but the pay-off is not guaranteed. For instance, according to [5], only the configuration of the six available parameters of kNN (k-Nearest Neighbors) requires about 17,000 settings in the context of software effort estimation (eight feature weighting methods, six similarity measures, five discretization methods, four adoption mechanisms, six analogies, and three case selectors).

Furthermore, many researchers believe that the default settings are always a better idea to implement because the development toolkits (e.g., R, Weka) might have come with the best default values, but in reality, it is not the case. The author of Weka (Eibe from Waikato University) mentioned that in an earlier version of Weka, the default number of trees in Random Forest was 10, which is now increased to a reasonable number 100, but still, it is not a perfect value (this value is 500 in R language) [6].

A. Background

Mendes [7] uses Bayesian Network with its default settings for building web effort estimation models with the Tukutuku dataset. Attarzadeh and Ow [8] propose the combination of Neural Network (NN) and COCOMO model with the default settings for software cost estimation. Similarly, Rijwani and Jain [9] employ Multi Layered Feed Forward Neural Network with the default settings for SDEE with the COCOMO dataset. Mende and Koschke [10] and Dyana et al. [11] employ default

settings in the application of the Random Forest (RF) for providing software defect predictions.

Also, as described in [1] for SFP and our analysis about SDEE, majority of the studies have used the default settings of the estimation techniques. However, researchers criticize the default parameter settings. For instance, Jiang et al. [12] and Tosun and Bener [13] describe that the default parameters provided by different development toolkits such as Weka [14], Python [15], R language [6], and Matlab [16] are suboptimal. Similarly, [17] states that the estimation techniques that are sensitive towards parameter optimization perform worst with the default values.

There are also studies which use different parameters optimization techniques to get the optimal settings. In particular, Can et al. [18] use Particle Swarm Optimization (PSO) to configure the parameters of SVM with JM1 dataset and Malhotra et al. [19] employ Simulated Annealing (SA) and Differential Evolution (DE) for parameter optimization in the context of SFP. Similarly, Tabu Search (TS) is employed to configure the parameters of SVR [20] and Genetic Programming (GP) is used for parameter optimization of Case-Based Reasoning (CBR) and StepWise Regression (SWR) in the domain of SDEE [21]. De Martino et al. [22] use GA to configure the parameters of SVM with jEdit data and the prediction accuracy achieved is better than the benchmark Grid search. Chakraborty et al. [23] provide an analysis about SFP classification-based datasets using Caret (Random Search and Grid Search) to evaluate which estimation techniques are more sensitive to parameter tuning and which are stable. In the extension of their work [24] they include GA and Differential Evolution (DE) along with the Caret's grid and random searches.

From the analysis of the above studies, we can note that it is worth interesting to assess the impact of parameter optimization and understand which optimization techniques should be used and which optimization techniques perform better when we have different types of data.

B. Aim of the paper and research questions

Our aim is to evaluate the impact/role of parameter optimization with various regression-based estimation techniques to verify: (i) how worth it is to optimize the setting of parameters, (ii) if the optimized settings of an estimation technique performs better with a particular type of data (e.g., SDEE) and if it provides better results with different type of data (e.g., SFP) too, (iii) if we need to optimize all the parameters of an estimation technique or there are some parameters that are insensitive towards optimization.

Prior studies have vague findings to conclude if the optimized settings have a positive impact on the prediction accuracy of software efforts and fault prediction models. A particular study in the SDEE domain has found that an estimation technique (e.g., SVM) is sensitive to parameters optimization [25] while another study in the domain of SFP, employing different experimental components (e.g., optimization algorithms, evaluation metrics, validation methods) suggests that SVM might not be sensitive to optimization [24].

To evaluate if the optimization of parameters has any positive impact on the prediction accuracy of the two important problems of software engineering (i.e., SDEE and SFP), we address the following research question:

RQ1: What is the impact of parameters optimization on the prediction accuracy in both SDEE and SFP contexts?

However, prior studies has also highlighted that the optimized settings of an estimation technique (e.g., SVM) which have a positive impact on SDEE data might not persist with SFP data, i.e., they are not transferable to different type of data [26] [27]. Another study by Gao et al. [28] has shown that the performance of estimation techniques is always data-specific.

To evaluate if the parameters optimization is sensitive to a specific type of data (either SDEE or SFP data), we formulate a second research question.

RQ2: Can optimal settings of parameters be transferred from one (type of SDEE and SFP) dataset to another?

To answer the two research questions, we exploit the results of an empirical study carried out by employing both SDEE and SFP data, nine estimation techniques that have been widely used in SDEE and SFP literature, and four different parameters optimization algorithms.

Structure of the paper. Section II presents the design of the performed study, while the results achieved and their discussion are presented in Section III. Conclusion and Future work conclude the paper.

II. STUDY DESIGN

A. Datasets

We consider six different datasets: three from the domain of SFP (Ant, Xalan, and Xerces) [10] and three from the domain of SDEE (China, Maxwell, and Nasa) [9]. The two types of datasets are completely different in nature. SFP datasets focus on the number of bugs detected at the class level while SDEE datasets have information about total effort (in person-month/person-hour) required for software development. Thus, SFP datasets have as target variable 'bug' while SDEE has 'effort' as dependent variable. All the datasets are publicly available regression-based datasets that are larger in size, both in terms of features and observations and, according to Mende [29], estimation techniques that are trained using larger datasets produce stable results. Similarly, Jiang et al. [30] illustrate that if small datasets are used for training models, the validation techniques do not perform better.

As shown in Table I, some of the important features of SFP datasets are WMC (Weighted Methods per Class), DIT (Depth of Inheritance Tree), CBO (Coupling Between Objects), NOC (Number of Children), and LOC (Lines of Code). These features are common to all the employed SFP datasets [31].

Software development effort (in person-months) is the output variable of all the three SDEE datasets. China dataset is developed by Chinese companies and has 499 observations and 19 features. Function types to calculate the number of Function Points, i.e., Input, Output, Inquiry, File, and Interface, represent the independent variables [31]. The Maxwell

dataset, developed by a commercial bank in Finland, includes information about 62 software projects in terms of 27 features. The important features include Function Points (SizeFP), T01 (customer participation), T02 (development environment adequacy), T03 (staff availability), T04 (standards used), T05 (methods used), T06 (tools used), T07 (software logical complexity), T08 (requirements volatility) [31]. The Nasa dataset includes 24 features describing information about 93 different projects by Nasa centers developed in the time period 1971-1987. It consists of 15 discrete independent variables such as rely (required software reliability), data (database size), cplx (process complexity), time (time constraint for cpu), stor (main memory constraint), virt (machine volatility), turn (turnaround time), acap (analyst capability), aexp (application experience), pcap (programmers capability). It also has the variable software size measured in KSLOC [31].

B. Parameter Optimization Algorithms

The majority of the studies that have employed parameter optimization algorithms, both in the domains of SDEE and SFP, have preferred to use the meta-heuristic algorithms [18] [19] [21] [22] [20]. In this study, we use four different optimization algorithms from different flavors, i.e., Grid Search (GS) and Random Search (RS) selected from the Caret package of R language [32], Simulated Annealing (SA) from the family of meta-heuristic algorithms [33], and Bayesian Optimization (BO) from the family of Gaussian Process [34].

1) *Simulated Annealing*: SA is used to find the best parameters in a search space, i.e., to search the space for all the possible values in order to find the global maximum likelihood [35]. The maximizing likelihood is difficult to analyze using mathematical methods such as derivations, whereas the implementation of SA in R language is specifically designed to analyze the likelihood functions. The procedure of SA for search space consists in walking a mountain range in the dark, searching for the highest mountain [19]. When the algorithm's temperature is high in the beginning of the search, the search process is energetic. The search goes up and down the mountain to find the best solution. But when the temperature and energy of the search get lower, the algorithm just moves the top to find the better path. The search (hopefully) finishes with the algorithm converging on the global maximum.

In R language, SA is implemented via a general-purpose optimization (optim) function [36], which has an option for SA that can be initialized as follow: optim (par, fn, method="SANN"), where par= initial parameters values to be optimized; and fn is a function to be minimized or maximized.

2) *Bayesian Optimization*: BO is one of those optimization algorithms that could be applied to a variety of problems, which makes use of a probabilistic model for the optimization of fitness functions [37] [38]. The advantage of BO is when the evaluation of the fitness function is expensive to conduct, e.g., the training of the machine learning techniques. It is easy to justify extra computation in order to get better decisions. In R language, BO is implemented with a specialized library, 'rBayesianOptimization'

for the parameter optimization that can be obtained from <https://github.com/yanyachen/rBayesianOptimization> [39]. For BO (and also for SA), we use 50 iterations to repeat the process. However, especially with BO, we can observe that the optimal parameter values are usually obtained after 15-25 iterations.

3) *Grid Search and Random Search*: The Caret package [32] offers two search algorithms, GS and RS to search and find the best parameters of an estimation technique. GS, represented as search="grid" in trainControl(), is the fundamental search technique based on an exhaustive search via a manually specified set of the parameters. GS systematically produces candidate parameter settings using the expand.grid method, evaluate each candidate parameter settings and identifies the optimal parameter settings. RS exhaustively searches via a randomly generated set of parameter space of a particular estimation technique. Since GS is computationally expensive and slow, the use of RS for finding the optimal parameters is more popular than GS [40]. Unlike, the predefined settings of parameters in GS, RS (represented as search='random' in trainControl) uses a parameter 'tuneLength' which defines the total number of parameter combinations needed to be evaluated. We use 30 as a parameter of tuneLength.

C. Validation method and evaluation criteria

We apply a (nested) k-fold CV because it is a widely employed validation method in Software Engineering (used in 49% studies, followed by hold-out validation in 45%, according to Chakkrit et al. [41]). Moreover, an empirical study performed to compare the performance of 10-fold CV with the Bootstrap validation method has found that the former performs better, and it was also less biased for small as well as large datasets [42]. As an inner resampling method, we apply 10-fold CV for obtaining best parameter values while 5-fold CV as an outer-resampling for performance evaluations.

To compare the prediction accuracy of different estimation models, we exploit the Mean Absolute Error (MAE) which is widely recommended by researchers [43] [44]. MAE evaluates how the predicted values differ from the actual values. It is also easy to compute:

$$MAE = \frac{\sum_{i=1}^n |p_i - a_i|}{n}$$

Moreover, to identify if a particular algorithm significantly outperform another, we use the paired samples Wilcoxon Signed Rank test (with $\alpha = 0.05$) [45]. The Wilcoxon test can be regarded as a safe test to use (even for normally distributed data), since it raises the bar for significance, by making no assumptions about underlying data distributions. A p-value less than 0.05 indicates that the null hypothesis (there is no difference between the absolute errors obtained with two models) can be rejected. If the p-value is greater than 0.05, we have no proof that a model has better performance than the other model.

Table I
INFORMATION ABOUT THE DATASETS

Datasets	Num. Features	Num. Obs.	Widely used features
Ant	16	745	WMC (Weighted Methods per Class), DIT (Depth of Inheritance Tree),
Xalan	21	885	CBO (Coupling Between Objects), NOC (Number of Children), and
Xerces	21	588	LOC (Lines of Code), CAM (Cohesion Among Methods of Class),
China	19	499	Input, Output, Inquiry, File, Interface
			Function Points (SizeFP), T01 (customer participation), T02 (development environment adequacy),
			T03 (staff availability), T04 (standards used), T05 (methods used), T06 (tools used),
Maxwell	27	62	T07 (software logical complexity), T08 (requirements volatility), T09 (quality requirements),
			T10 (efficiency requirements), T11 (installation requirements), T12 (staff analysis skills),
			T13 (staff application knowledge), T14 (staff tool skills), and T15 (staff team skill).
			rely (required software reliability), data (database size), cplx (process complexity),
			time (time constraint for cpu), stor (main memory constraint), virt (machine volatility),
Nasa	24	93	turn (turnaround time), acap (analyst capability), aexp (application experience),
			pcap (programmers capability), vexp (virtual machine experience), lexp (language experience),
			modp (modern programing practices), tool (use of software tools), sced (schedule constraint), KSLOC

Furthermore, to quantify the degree of improvement of various estimation techniques, we employ Vargha and Delaney effect size [46]. It is to be noted that we do not consider here the pooled Cohen's effect size because it requires the data to be normalized. Vargha and Delaney effect size can be represented as: $A12 = (R1/m - (m+1)/2)/n$, where R1 is the rank sum of the first group of samples whereas m and n are the observations of both samples [47]. Thus, the A12 statistics quantify the probability that using method A yields better MAE than using another method B. If A and B are equivalent then $A12=0.5$. If A performs better than B, the effect size can be considered as small if A12 is from 0.57- 0.64, medium if A12 is from 0.65 to 0.71, and large if A12 is more than 0.71. The effect size is negligible if A12 is equals or less than 0.56.

D. Threats to validity

There are indeed some factors that can bias the validity of the performed empirical analysis.

Construct validity. We use regression-based SFP datasets, which could have different results than the classification-based datasets. However, to provide a different type of datasets and strengthen the generalization of our results, we also employed SDEE datasets. Hence, our findings are not restricted to just a specific type of input metrics.

External validity. The number of estimation techniques used in regression problems are less than the one employed in classification-based problems. We employ estimation techniques implemented in the R language that have been widely used in different SDEE studies and regression-based SFP problems. However, some least frequently used regression-based estimation techniques might produce different results, but they are not frequent in the domain of SFP and SDEE. Moreover, we are aware that in the context of SFP and SDEE studies, GA and PSO are the algorithms more employed compared to other parameter optimization algorithms and our results could be different if we used those algorithms in our work. In the future, we could extend also the list of parameter optimization algorithms, including meta-heuristic algorithms.

III. RESULTS AND DISCUSSION

In this section, we present and discuss the results of our study considering each research question.

A. RQ1: What is the impact of parameters optimization on the prediction accuracy in both SDEE and SFP contexts?

We have considered 9 estimation techniques, 4 parameter optimization algorithms and 1 default parameters settings, and 3 datasets for the SDEE problem, hence we have a total of $9 \times 4 \times 3 = 108$ different experiments, where we compare prediction accuracy achieved with the optimized settings and the default settings (using the R development toolkit).

Figure 1 shows the experiments where the optimized settings or the default settings provided better results as well as the number of experiments where no difference can be observed between the obtained predictions. The number of experiments where the individual parameter optimization algorithms have provided better results are also shown.

The results reveal that for all the estimation techniques (except for NNET and avNNET), at least one parameter optimization algorithm performs better than the default settings in at least two of the three considered datasets. Note that, we have employed three different flavors of Neural Networks included in the R packages, i.e., NNET, avNNET, and MLP and the results suggest that the parameter optimization is characterized by better results than the default settings only with MLP. Thus, it seems that NNET and avNNET's parameters (size, weight decay, and bagging) are not sensitive towards parameter optimization. It means that both the weight decay and size parameters (of NNET and avNNET) do not result to be parameter sensitive while the size, which is the only parameter of MLP, could be regarded as sensitive as we see that the BO has performed better with MLP than the default settings with a notable effect size.

Overall, we can also highlight that for the SDEE domain seven out of nine estimation techniques (78%) require optimization of at least one parameter, that eventually leads to provide better prediction accuracy than the estimation techniques trained with the default settings.

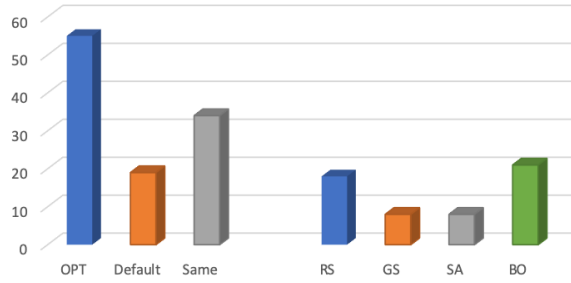


Figure 1. Performance evaluations with SDEE datasets, where OPT means optimized settings performed better, Default means default settings performed better, and Same means optimized and default settings have same performance

Furthermore, focusing on the experiments where the optimization algorithms provided better performance, BO is the most effective algorithm, providing better predictions in all the experiments with the estimation techniques that are sensitive to parameter optimization (21/21, i.e., 7 parameter sensitive estimation techniques * 3 datasets=21) and a notable effect size, of either small, medium or large, can be observed in 19 (90%) of experiments. The second best-performed algorithm is the Caret's RS, which has provided better results in 18 experiments (with a notable effect size observed in 14, i.e., 77%, experiments).

Regarding the other optimization algorithms, in the majority of the experiments, GS has the same performance as the default settings, while the worst performed optimization algorithm is SA, which has been outperformed by the default settings in the majority of the experiments.

With SFP data we have a total of 108 experiments too and differently from above the optimized settings of NN and avNNET obtained better predictions than the default settings. Thus, we can consider these techniques sensitive to the optimization with SFP data. GBM and PLS have no, or less, significant influence with parameter optimization. Only BO has (partially) performed better with these two techniques but with no notable effect size. kNN with the BO has also performed better in two out of three datasets, with a small effect size. Thus, we can consider kNN (k parameter) as sensitive towards parameter optimization.

Overall, with SFP datasets, we can highlight that seven out of nine estimation techniques (excluding GBM and PLS) require optimization of at least one parameter.

Figure 2 shows a summary of performed comparisons by showing the number of cases where the optimized and default settings performed better. We can see that again, BO is the best performing optimization algorithm (among those considered), providing better results in 19 out of 21 (7 parameter sensitive estimation techniques * 3 datasets) experiments (90%) with the estimation techniques having sensitive parameters. In these 19 experiments, we can observe a notable effect size for 15 experiments/comparisons. Similar to the SDEE experiments, the second best performed optimization algorithm is RS, providing

better results in 13 out of 21 experiments (62%), with a notable effect size in all the 13 experiments. Likewise, SDEE, GS, and SA have no satisfactory performances, providing better results in 11 and 8 experiments, respectively.

In summary, we can positively answer RQ1 because at least one parameter optimization algorithm has allowed to obtain better predictions than the default settings in at least two out of three datasets, with an effect size of either small, medium, or large.

Key findings The estimation techniques are sensitive to parameter optimization for both types of datasets (i.e., SDEE and SFP).

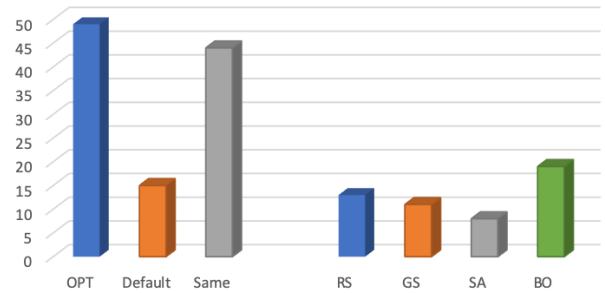


Figure 2. Performance evaluations with SFP datasets, where OPT means optimized settings performed better, Default means default settings performed better, and Same means optimized and default settings have same performance

B. RQ2: Can optimal settings of parameters be transferred from one (type of SDEE and SFP) dataset to another?

The analysis conducted to answer RQ1 has revealed that for both the SDEE and SFP datasets seven out of nine estimation techniques (78%) require at least one parameter to be optimized for providing better predictions than the default settings. If an estimation technique with its parameter settings performs better than the default settings, for one type of data (e.g., China for SDEE) and then it also performs better with another dataset of the same type (e.g., Maxwell for SDEE), we can say that the (sensitive) parameters of that estimation technique can be transferred to the data of similar type, i.e., data with same input metrics. Similarly, if the same estimation technique with its optimized parameter values also performs better with a different type of data (e.g., Xerces for SFP), then we can say that the (sensitive) parameters of the estimation techniques can be transferred to the same data as well as to data of different type, i.e., data having different input metrics.

To address our RQ2, we need to evaluate which estimation technique's parameters are transferable to data of similar type (i.e., SDEE or SFP) and which are transferable even to data of different types (i.e., from SDEE to SFP or from SFP to SDEE).

All the nine estimation techniques we have employed have 16 different parameters in total. Since NNet and avNNet (together they have five parameters) do not perform better with

SDEE datasets, we can say that their parameter optimizations are not transferable to SDEE datasets. Same as the case for the SFP datasets, where PLS and GBM (together they have also five parameters) do not manage to perform better than the default settings and their parameter optimizations cannot be transferred to the SFP datasets.

Therefore, we can say that, for both SDEE and SFP datasets, 10 out of 16 parameters (62%) of the employed estimation techniques could be transferred to data of the same type (i.e., with similar input metrics) with a notable effect size of either small, medium, or large. Similarly, since the estimation techniques CART, KNN, RF, MLP, and SVM have performed better with the optimized settings both for the SDEE and SFP datasets and together they have six parameters (i.e., CART=1, KNN=1, RF=1, MLP=1, and SVM=2), we can also say that only six out of 16 (37%) parameters are transferred to data of same as well as of completely different type, with a notable effect size.

Thus, our answer to the second research question can be summarized as follow: 10 out of 16 parameters are sensitive to only a specific type of data (either SDEE or SFP) and only 6 parameters are transferable to data of same as well as completely different type (both SDEE and SFP data).

As also suggested by the no free lunch theorem [48], a particular (optimization) algorithm might not work better in all the cases. We cannot just pick the best-performed optimization algorithm from the literature and apply to our data because it has been empirically proved that there are less chances to transfer optimal parameter settings from one type of data to another.

C. Identification of sensitive parameters

Each estimation technique has a few parameters and the values of those parameters control to a large extent the prediction accuracy of the estimation technique. For instance, `mtry` is the important parameter of RF and its value must be optimized/configured to some (optimal) values in order to provide the best prediction accuracy for RF. However, not all parameters of an estimation technique are sensitive to parameter optimization. Sensitive parameters are those parameters of an estimation technique that have a major impact/role on the prediction accuracy compared to the other parameters. Sensitive parameters of an estimation technique need to be optimized/configured (using parameter optimization algorithms) in order to provide accurate predictions.

Identifying the most sensitive parameters of an estimation technique is highly recommended so that the researchers/practitioners can only focus on the optimization of parameters that would allow to obtain more accurate predictions than the other (non-sensitive) parameters. Thus, we have quantified the influence and the impact of each parameter of an estimation technique in our empirical study. To this end, we first train an estimation technique with all of its parameters, by setting them to their default settings, apart from the parameter whose effect we want to evaluate, which is set to its optimized/configured settings. We repeat these steps

for each parameter of an estimation technique and hence we identify the sensitive parameters for that estimation technique. The estimation technique provides more worst predictions when its sensitive parameters are not optimized (i.e., they are set to their default settings).

Though for different optimization algorithms we have got slightly different results, in overall cases, we have found that `sigma` is a more sensitive parameter for SVM with both the SDEE and SFP datasets. Similarly, `size` is observed as a more sensitive parameter than `weight decay` for NNet with both types of data, and the Boolean parameter `bagging` is observed as more sensitive than the `size` and `weight decay` parameters for `avNNet`. Regarding the parameters of GBM, we have found that `Min_terminal Node.size` is more sensitive when used with the SDEE datasets and `tree depth` is reported as more sensitive than the other parameters for SFP data. Again, we want to emphasize that in a very few experiments, some exceptions are applied to our findings. For instance, in a few cases, we found that `weight` is more sensitive to optimization than the `size` for NNET. However, these are very few experiments.

We can see the glimpse of these sensitive parameters in Figure 3 and Figure 4, for SDEE and SFP data, respectively. Due to the space constraints, we only select one case/experiment from SDEE data (Figure 3) and one from SFP data (Figure 4). We have selected BO and RS because these two were the best-performed optimization algorithms among the four employed.

Key findings Finding sensitive parameters of a particular estimation technique allows to only optimize those parameters that will improve the prediction accuracy rather than optimizing all the parameters. As optimization takes a huge amount of time, it will save our resources.

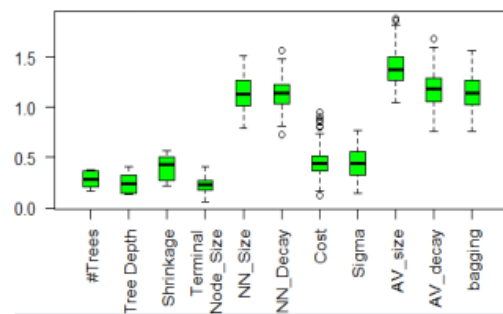


Figure 3. Sensitive parameters for RS/China

IV. CONCLUSION AND FUTURE WORK

In this work, we have evaluated the overall influence of parameter optimization on the estimation techniques that are used in the domains of SDEE and SFP. We tried to provide a detailed empirical study to determine in which cases the optimization of parameters pay-off, both in the context of SEE and SFP. It has been found that the optimization of NN

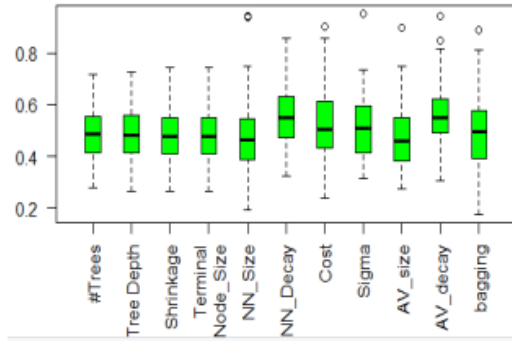


Figure 4. Sensitive parameters for BO/Ant

and avNNET parameters with SDEE data and PLS & GBM's parameters with SFP data does not improve the prediction accuracy. Also, not all parameters of an estimation technique are sensitive to optimization and before applying some optimization techniques, identifying of sensitive parameters are recommended.

In the future, we intend to evaluate how stable (variability of performance estimates when experiments are repeated) are the estimation techniques before the optimization and how optimization impacts the stability ratio of the estimation techniques. If estimation techniques are not stable, conclusions could not be drawn from SDEE/SFP studies [49] [50].

REFERENCES

- [1] T. Menzies, A. Butcher, D. R. Cok, A. Marcus, L. Layman, F. Shull, B. Turhan, and T. Zimmermann, "Local versus global lessons for defect prediction and effort estimation," *IEEE Trans. Software Eng.*, vol. 39, no. 6, pp. 822–834, 2013.
- [2] T. Menzies, D. Port, Z. Chen, J. Hihn, and S. Stukes, "Validation methods for calibrating software effort models," in *27th International Conference on Software Engineering (ICSE 2005), 15-21 May 2005, St. Louis, Missouri, USA*, G. Roman, W. G. Griswold, and B. Nuseibeh, Eds. ACM, 2005, pp. 587–595.
- [3] W. Fu, T. Menzies, and X. Shen, "Tuning for software analytics: Is it really necessary?" *Inf. Softw. Technol.*, vol. 76, pp. 135–146, 2016.
- [4] A. Ali and C. Gravino, "A systematic literature review of software effort prediction using machine learning methods," *J. Softw. Evol. Process.*, vol. 31, no. 10, 2019.
- [5] E. Kocaguneli, T. Menzies, A. Bener, and J. W. Keung, "Exploiting the essential assumptions of analogy-based effort estimation," *IEEE Trans. Software Eng.*, vol. 38, no. 2, pp. 425–438, 2012.
- [6] R. D. C. Team, R: A language and environment for statistical computing, R Foundation for Statistical Computing, Vienna, Austria, 2010. [Online]. Available: <http://www.r-project.org>
- [7] E. Mendes, "The use of a bayesian network for web effort estimation," in *Web Engineering, 7th International Conference, ICWE 2007, Como, Italy, July 16-20, 2007, Proceedings*, ser. Lecture Notes in Computer Science, L. Baresi, P. Fraternali, and G. Houben, Eds., vol. 4607. Springer, 2007, pp. 90–104.
- [8] I. Attarzadeh and S. H. Ow, "Proposing an effective artificial neural network architecture to improve the precision of software cost estimation model," *Int. J. Softw. Eng. Knowl. Eng.*, vol. 24, no. 6, pp. 935–954, 2014. [Online]. Available: <https://doi.org/10.1142/S0218194014500338>
- [9] P. Rijwani and S. Jain, "Enhanced software effort estimation using multi layered feed forward artificial neural network technique," *Procedia Computer Science*, vol. 89, pp. 307–312, 2016.
- [10] T. Mende and R. Koschke, "Revisiting the evaluation of defect prediction models," in *Proceedings of the 5th International Workshop on Predictive Models in Software Engineering, PROMISE 2009, Vancouver, BC, Canada, May 18-19, 2009*, T. J. Ostrand, Ed. ACM, 2009, p. 7. [Online]. Available: <https://doi.org/10.1145/1540438.1540448>
- [11] D. R. Ibrahim, R. Ghnemat, and A. Hudaib, "Software defect prediction using feature selection and random forest algorithm," in *2017 International Conference on New Trends in Computing Sciences (ICTCS)*, 2017, pp. 252–257.
- [12] Y. Jiang, B. Cukic, and T. Menzies, "Can data transformation help in the detection of fault-prone modules?" in *Proceedings of the 2008 Workshop on Defects in Large Software Systems, held in conjunction with the ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2008), DEFECTS 2008, Seattle, Washington, USA, July 20, 2008*, P. T. Devanbu, B. Murphy, N. Nagappan, and T. Zimmermann, Eds. ACM, 2008, pp. 16–20. [Online]. Available: <https://doi.org/10.1145/1390817.1390822>
- [13] A. Tosun and A. B. Bener, "Reducing false alarms in software defect prediction by decision threshold optimization," in *Proceedings of the Third International Symposium on Empirical Software Engineering and Measurement, ESEM 2009, October 15-16, 2009, Lake Buena Vista, Florida, USA*. IEEE Computer Society, 2009, pp. 477–480. [Online]. Available: <https://doi.org/10.1109/ESEM.2009.5316006>
- [14] M. Hall, E. Frank, G. Holmes, B. Pfahringer, P. Reutemann, and I. H. Witten, "The weka data mining software: An update," *SIGKDD Explor. Newsl.*, vol. 11, no. 1, p. 10–18, nov 2009. [Online]. Available: <https://doi.org/10.1145/1656274.1656278>
- [15] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg et al., "Scikit-learn: Machine learning in python," *Journal of Machine Learning Research*, vol. 12, no. Oct, pp. 2825–2830, 2011.
- [16] MATLAB, version 8.5.0 (R2010a). Natick, Massachusetts: The MathWorks Inc., 2015.
- [17] T. Hall, S. Beecham, D. Bowes, D. Gray, and S. Counsell, "A systematic literature review on fault prediction performance in software engineering," *IEEE Transactions on Software Engineering*, vol. 38, no. 6, pp. 1276–1304, 2012.
- [18] H. Can, X. Jianchun, Z. Ruide, L. Juelong, Y. Qiliang, and X. Liqiang, "A new model for software defect prediction using particle swarm optimization and support vector machine," in *2013 25th Chinese Control and Decision Conference (CCDC)*, 2013, pp. 4106–4110.
- [19] R. Malhotra, A. Rajpal, and D. Rathore, "Parameter tuning on software defect prediction using differential evolution amp; simulated annealing," in *2018 International Conference on Big Data and Artificial Intelligence (BDAl)*, 2018, pp. 97–106.
- [20] A. Corazza, S. D. Martino, F. Ferrucci, C. Gravino, F. Sarro, and E. Mendes, "How effective is tabu search to configure support vector regression for effort estimation?" in *Proceedings of the 6th International Conference on Predictive Models in Software Engineering, PROMISE 2010, Timisoara, Romania, September 12-13, 2010*, T. Menzies and G. Koru, Eds. ACM, 2010, p. 4. [Online]. Available: <https://doi.org/10.1145/1868328.1868335>
- [21] F. Ferrucci, C. Gravino, R. Oliveto, and F. Sarro, "Genetic programming for effort estimation: An analysis of the impact of different fitness functions," in *2nd International Symposium on Search Based Software Engineering*, 2010, pp. 89–98.
- [22] S. D. Martino, F. Ferrucci, C. Gravino, and F. Sarro, "A genetic algorithm to configure support vector machines for predicting fault-prone components," in *Product-Focused Software Process Improvement - 12th International Conference, PROFES 2011, Torre Canne, Italy, June 20-22, 2011. Proceedings*, ser. Lecture Notes in Business Information Processing, D. Caivano, M. Oivo, M. T. Baldassarre, and G. Visaggio, Eds., vol. 6759. Springer, 2011, pp. 247–261. [Online]. Available: https://doi.org/10.1007/978-3-642-21843-9_20
- [23] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, "Automated parameter optimization of classification techniques for defect prediction models," in *2016 IEEE/ACM 38th International Conference on Software Engineering (ICSE)*, 2016, pp. 321–332.
- [24] —, "The impact of automated parameter optimization on defect prediction models," *IEEE Transactions on Software Engineering*, vol. 45, no. 7, pp. 683–711, 2019.
- [25] A. L. Oliveira, P. L. Braga, R. M. Lima, and M. L. Cornélio, "Ga-based method for feature selection and parameters optimization for machine

- learning regression applied to software effort estimation,” *information and Software Technology*, vol. 52, no. 11, pp. 1155–1166, 2010.
- [26] B. Ghotra, S. McIntosh, and A. E. Hassan, “Revisiting the impact of classification techniques on the performance of defect prediction models,” in *2015 IEEE/ACM 37th IEEE International Conference on Software Engineering*, vol. 1, 2015, pp. 789–800.
- [27] T. Khoshgoftaar and N. Seliya, “Comparative assessment of software quality classification techniques: An empirical case study,” *Empirical Software Engineering*, vol. 9, pp. 229–257, 09 2004.
- [28] K. Gao, T. Khoshgoftaar, H. Wang, and N. Seliya, “Choosing software metrics for defect prediction: An investigation on feature selection techniques,” *Softw., Pract. Exper.*, vol. 41, pp. 579–606, 04 2011.
- [29] T. Mende, “Replication of defect prediction studies: problems, pitfalls and recommendations,” in *Proceedings of the 6th International Conference on Predictive Models in Software Engineering, PROMISE 2010, Timisoara, Romania, September 12-13, 2010*, T. Menzies and G. Koru, Eds. ACM, 2010, p. 5.
- [30] Y. Jiang, J. Lin, B. Cukic, and T. Menzies, “Variance analysis in software fault prediction models,” in *ISSRE 2009, 20th International Symposium on Software Reliability Engineering, Mysuru, Karnataka, India, 16-19 November 2009*. IEEE Computer Society, 2009, pp. 99–108.
- [31] A. Ali and C. Gravino, “Evaluating the impact of feature selection consistency in software prediction,” *Science of Computer Programming*, vol. 213, p. 102715, 2022.
- [32] M. Kuhn, “Building predictive models in r using the caret package,” *Journal of Statistical Software*, vol. 28, no. 5, p. 1–26, 2008.
- [33] K. Husmann, A. Lange, and E. Spiegel, “The r package optimization: Flexible global optimization with simulated-annealing,” 2017.
- [34] O. Adesina, A. S. Onanaye, and D. Okewole, “Bayesian optimization for parameter of discrete weibull regression,” *Journal of Advances in Mathematics and Computer Science*, vol. 34, pp. 1–13, 01 2020.
- [35] <https://www.rdocumentation.org/packages.../optim>.
- [36] Y. Xiang, S. Gubian, B. P. Suomela, and J. Hoeng, “Generalized simulated annealing for global optimization: The gensa package,” *R J.*, vol. 5, p. 13, 2013.
- [37] J. Snoek, H. Larochelle, and R. P. Adams, “Practical bayesian optimization of machine learning algorithms,” ser. NIPS’12, 2012, p. 2951–2959.
- [38] P. Frazier, “A tutorial on bayesian optimization,” 2018.
- [39] Y. Yan, “Package rbayesianoptimization.”
- [40] R. G. Mantovani, A. L. D. Rossi, J. Vanschoren, B. Bischl, and A. C. P. L. F. de Carvalho, “Effectiveness of random search in SVM hyper-parameter tuning,” in *2015 International Joint Conference on Neural Networks, IJCNN 2015, Killarney, Ireland, July 12-17, 2015*. IEEE, 2015, pp. 1–8. [Online]. Available: <https://doi.org/10.1109/IJCNN.2015.7280664>
- [41] C. Tantithamthavorn, S. McIntosh, A. E. Hassan, and K. Matsumoto, “An empirical comparison of model validation techniques for defect prediction models,” *IEEE Trans. Software Eng.*, vol. 43, no. 1, pp. 1–18, 2017. [Online]. Available: <https://doi.org/10.1109/TSE.2016.2584050>
- [42] J.-H. Kim, “Estimating classification error rate: Repeated cross-validation, repeated hold-out and bootstrap,” *Computational Statistics Data Analysis*, vol. 53, no. 11, pp. 3735–3745, 2009. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0167947309001601>
- [43] M. J. Shepperd and S. G. MacDonell, “Evaluating prediction systems in software project estimation,” *Inf. Softw. Technol.*, vol. 54, no. 8, pp. 820–827, 2012. [Online]. Available: <https://doi.org/10.1016/j.infsof.2011.12.008>
- [44] W. B. Langdon, J. J. Dolado, F. Sarro, and M. Harman, “Exact mean absolute error of baseline predictor, MARP0,” *Inf. Softw. Technol.*, vol. 73, pp. 16–18, 2016. [Online]. Available: <https://doi.org/10.1016/j.infsof.2016.01.003>
- [45] B. A. Kitchenham, L. Pickard, S. G. MacDonell, and M. J. Shepperd, “What accuracy statistics really measure,” *IEE Proc. Softw.*, vol. 148, no. 3, pp. 81–85, 2001. [Online]. Available: <https://doi.org/10.1049/ip-sen:20010506>
- [46] A. Vargha and H. D. Delaney, “A critique and improvement of the cl common language effect size statistics of mcgraw and wong,” *Journal of Educational and Behavioral Statistics*, vol. 25, no. 2, pp. 101–132, 2000.
- [47] F. Sarro, A. Petrozziello, and M. Harman, “Multi-objective software effort estimation,” in *Proceedings of the 38th International Conference on Software Engineering*, ser. ICSE ’16. New York, NY, USA: Association for Computing Machinery, 2016, p. 619–630. [Online]. Available: <https://doi.org/10.1145/2884781.2884830>
- [48] D. H. Wolpert and W. G. Macready, “No free lunch theorems for optimization,” *IEEE transactions on evolutionary computation*, vol. 1, no. 1, pp. 67–82, 1997.
- [49] C. Mair and M. J. Shepperd, “The consistency of empirical comparisons of regression and analogy-based software project cost prediction,” in *2005 International Symposium on Empirical Software Engineering (ISESE 2005), 17-18 November 2005, Noosa Heads, Australia*. IEEE Computer Society, 2005, pp. 509–518.
- [50] I. Myrtveit, E. Stensrud, and M. J. Shepperd, “Reliability and validity in comparative studies of software prediction models,” *IEEE Trans. Software Eng.*, vol. 31, no. 5, pp. 380–391, 2005.